

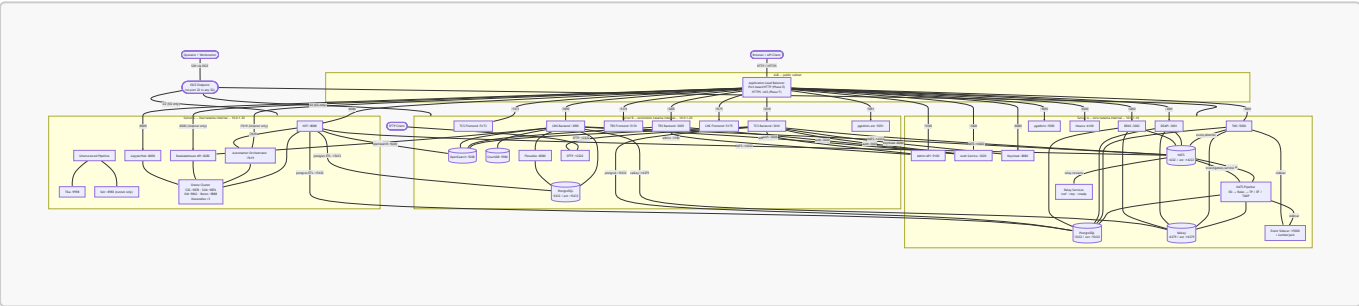
End-to-End Service Flow

This document maps all inter-service communication across the three-server Tazama AWS deployment. Each section below covers a distinct data path or functional area.

Server	Hostname	Private IP	Stack
Server A	core.tazama.internal	10.0.1.10	tazama-core
Server B	extensions.tazama.internal	10.0.1.20	tazama-extensions
Server C	biar.tazama.internal	10.0.1.30	tazama-biar

1. System Architecture Overview

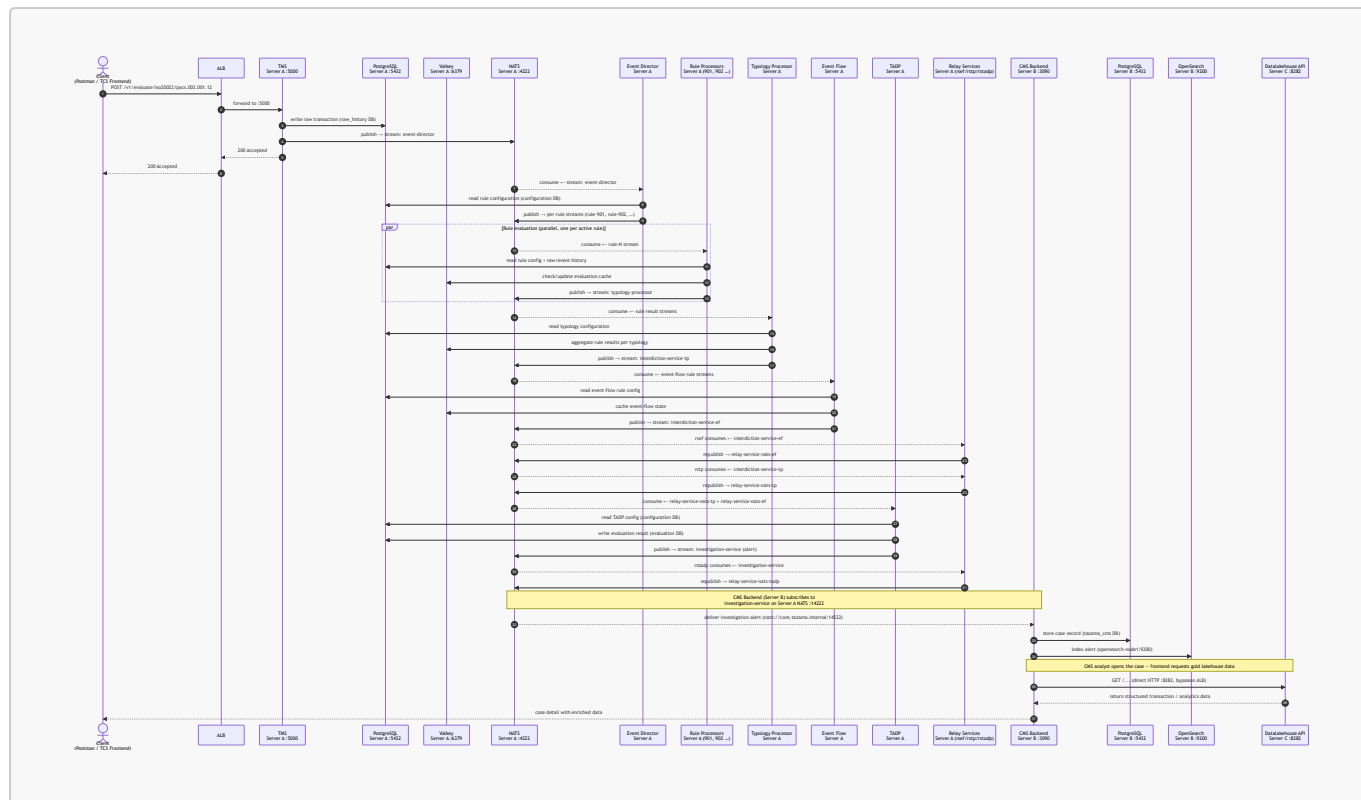
All three servers sit in a private subnet with no public IPs. User traffic enters via the Application Load Balancer (public subnet). Operator SSH access uses the EC2 Instance Connect Endpoint (EICE) — no port 22 exposed to the internet.



2. Transaction Processing Pipeline

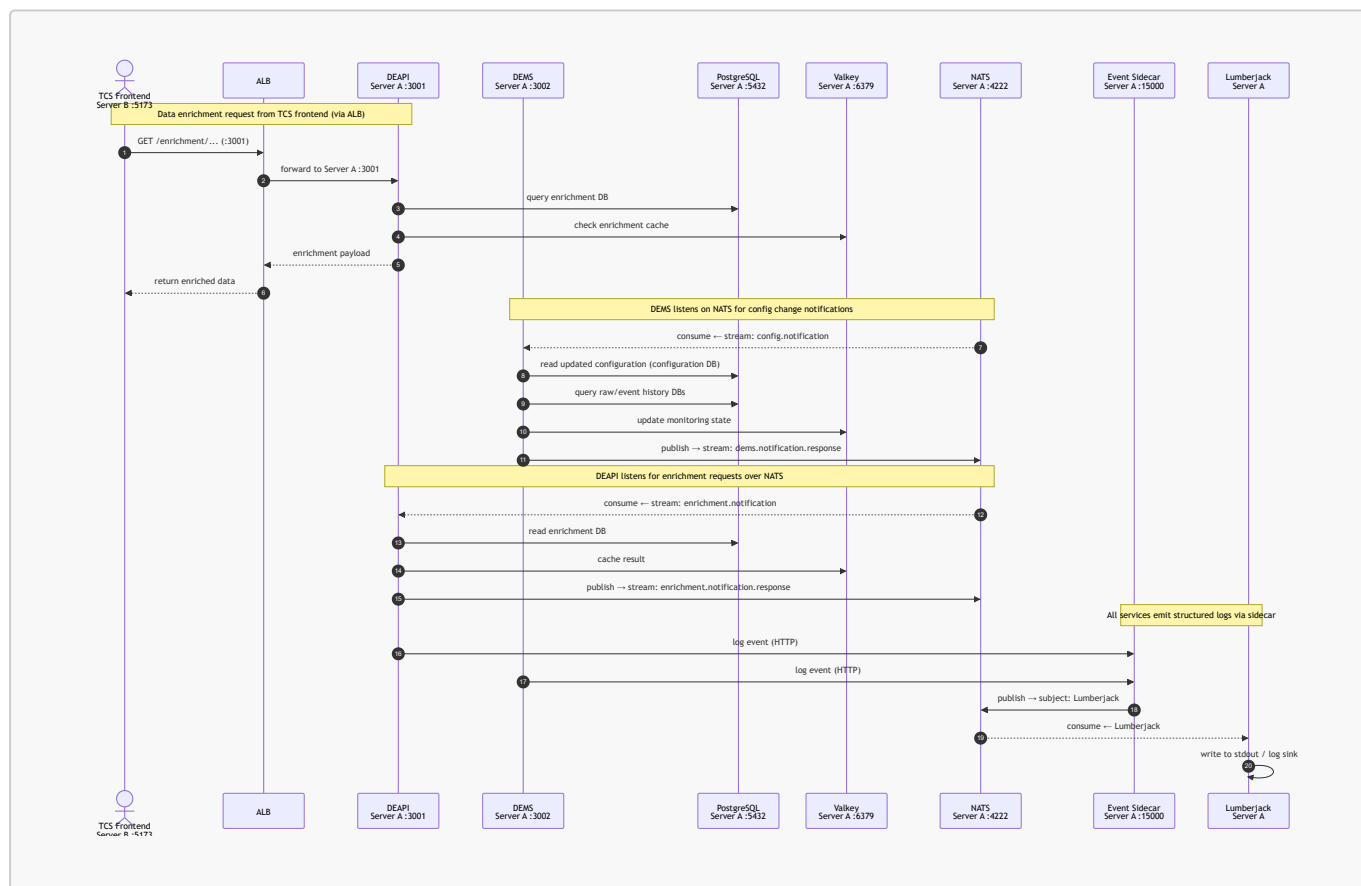
The core Tazama flow: a financial message is submitted to the TMS, fanned out through the NATS-based rule evaluation pipeline, decisioned by TADP, and the resulting investigation alert is consumed by the CMS on Server B.

DEAPI and DEMS run inside the `tazama-core` Docker project on **Server A** (launched as a pre-flight step for extensions) so they share the internal Docker network with NATS, postgres, and valkey. Their exterior ports (:3001, :3002) allow external callers (Server B frontends via ALB) to reach them across the network.



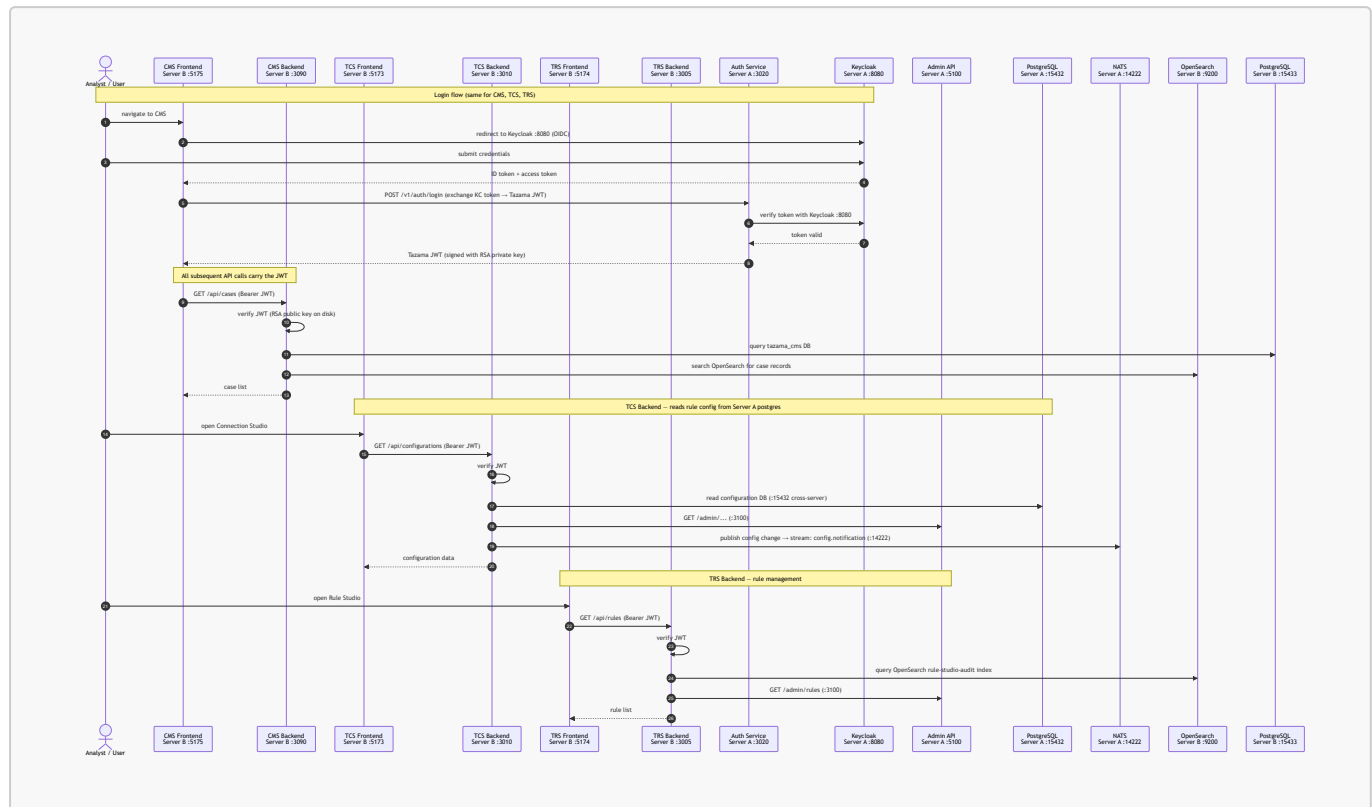
3. Data Enrichment and Event Monitoring (DEAPI / DEMS)

DEAPI and DEMS are both co-hosted on **Server A** (tazama-core project). They are called by frontends via the ALB and by the NATS pipeline for enrichment callbacks.



4. Tooling Frontend Authentication and API Flows

TCS, TRS, and CMS frontends follow the same auth pattern: the frontend requests a JWT from the Auth Service (via the TMS auth endpoint or Keycloak), then the backend validates the JWT locally using the RSA public key.



5. BIAR Data Pipeline (NiFi ETL and Analytics)

NiFi orchestrates data movement between Server A's PostgreSQL, Server B's PostgreSQL and OpenSearch, and the Ozone object store on Server C. The Datalakehouse API exposes the processed (gold) data to the CMS backend.



Server B → Server A

Server B → Server C

Source service	Destination	Port	Protocol	Purpose
cms-backend	Datalakehouse API	8282	TCP/HTTP	Gold lakehouse data (direct, not via ALB)

Server C → Server A

Source service	Destination	Port	Protocol	Purpose
NiFi	PostgreSQL (Server A)	15432	TCP/JDBC	ETL extraction (raw_history, event_history, evaluation)
NiFi	NATS	14222	TCP/NATS	Pipeline integration

Server C → Server B

Source service	Destination	Port	Protocol	Purpose
NiFi	PostgreSQL (Server B)	15433	TCP/JDBC	ETL extraction (tazama_cms, tazama_dwh)
NiFi	OpenSearch	9200	TCP/HTTP	ETL extraction (alert / audit indices)

ALB → Servers (per-server, current port-based HTTP routing)

Ports with † have ALB listener target groups configured but the port is **not** currently open in the ALB Security Group — those services are accessed via SSH tunnel (Phase E.1). They will be opened to the internet when promoted to Phase F subdomain routing.

Target	Port	Service
Server A	5000	TMS API
Server A	3001	DEAPI
Server A	3002	DEMS
Server A	3020	Auth Service
Server A	5100	Admin API
Server A	8080	Keycloak
Server A	5050	pgAdmin (core)
Server A	6100	Hasura
Server B	3005	TRS Backend
Server B	3010	TCS Backend
Server B	3090	CMS Backend
Server B	5051	pgAdmin (extensions)

Target	Port	Service
Server B	5173	TCS Frontend
Server B	5174	TRS Frontend
Server B	5175	CMS Frontend
Server C	8088	NiFi UI
Server C	8000 †	JupyterHub
Server C	7619 †	Automation Orchestrator
Server C	8282 †	Datalakehouse API

Operator-only (SSH tunnel via EICE, no SG inbound rules)

Server	Port	Service
Server C	8983	Solr UI
Server C	9876	Ozone SCM
Server C	9878	Ozone S3G
Server C	9888	Ozone Recon UI

7. NATS Stream Map (Server A)

The full NATS message path through the Server A pipeline for a single transaction evaluation.

